

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
“ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”

Кафедра “Програмної інженерії та інтелектуальних технологій управління”

ЗВІТ

З лабораторної роботи №2

За темою: «**Робота з переліками та структурами**»

Варіант 13

Виконав студент
Групи КН-224Б
Макаренко Максим Олександрович

Перевірив
Василенко Артем Вікторович

Харків-2025

Хід роботи

1.1 Перелік для представлення місяців року

Створити перелік для представлення місяців року. Реалізувати та продемонструвати перевантаження операцій `++` і `--` так, щоб після грудня йшов січень, а перед січнем – грудень.

main.cpp

```
#include <iostream>
using namespace std;

enum Month { January, February, March, April, May, June, July, August, September,
October, November, December };

Month& operator++(Month& m) {
    m = (m == December) ? January : Month(m + 1);
    return m;
}

Month& operator--(Month& m) {
    m = (m == January) ? December : Month(m - 1);
    return m;
}

std::string getMonthName(Month m) {
    const string names[] =
{"January", "February", "March", "April", "May", "June", "July", "August", "September", "October",
"November", "December"};
    return names[m];
}

int main(void) {
    Month m = December;
    ++m;
    cout << "Month after December: " << getMonthName(m) << "\n";
    --m;
    cout << "Months before January: " << getMonthName(m) << "\n";
    return 0;
}
```

Додатковий опис:

1.Результат роботи:

```
● maksim@192 1.1 % g++ -std=c++17 main.cpp -o main && ./main  
  
Month after December: January  
Months before January: December
```

Рисунок 1 — результат коректної роботи програми

1.2 Точки в тривимірному просторі

Створити структуру для опису точки в тривимірному просторі (тип точки). Написати програму, яка обчислює відстань між двома точками. Для обчислення відстані створити функцію з двома параметрами типу структури для представлення точок.

main.cpp

```
#include <iostream>  
#include <cmath>  
using namespace std;  
  
struct Point3D {  
    double x, y, z;  
};  
  
double distance(const Point3D& p1, const Point3D& p2) {  
    return sqrt(pow(p2.x - p1.x, 2) + pow(p2.y - p1.y, 2) + pow(p2.z - p1.z, 2));  
}  
  
int main(void) {  
    Point3D p1, p2;  
  
    cout << "Введіть координати першої точки (x y z): ";  
    cin >> p1.x >> p1.y >> p1.z;  
  
    cout << "Введіть координати другої точки (x y z): ";  
    cin >> p2.x >> p2.y >> p2.z;  
  
    cout << "Відстань між точками: " << distance(p1, p2) << "\n";  
  
    return 0;  
}
```

Додатковий опис:

1.Результат роботи:

```
● maksim@192 1.2 % g++ -std=c++17 main.cpp -o main && ./main  
  
Введіть координати першої точки (x y z): 2 6 9  
Введіть координати другої точки (x y z): 1 11 6  
Відстань між точками: 5.91608
```

Рисунок 2 — результат коректної роботи програми

1.3 Представлення й обробка даних про студентів у масиві

Створити структуру для представлення даних про студента. Структура повинна містити такі елементи даних:

- номер студентського посвідчення (**unsigned int**);
- прізвище (рядок у вигляді масиву символів);
- оцінки за останню сесію у вигляді масиву цілих від 0 до 100 (оцінки за предметами).

Реалізувати функцію яка здійснює виведення даних про студента у консольне вікно. Першим параметром функції повинна бути структура, яка описує студента.

Реалізувати функції, які отримують масив вказівників на студентів і довжину масиву і здійснюють

- сортування масиву за ознакою, яка наведена в індивідуальному завданні;
- пошук даних про студентів, які відповідають умові, наведеній в індивідуальному завданні.
- виведення всіх елементів масиву в консольне вікно.

Під час сортування масиву структур замість обміну місцями двох структур здійснювати обмін місцями елементів масиву вказівників.

Створити масив студентів. Створити масив вказівників на студентів, заповнивши його адресами структур з масиву студентів. Продемонструвати сортування й пошук студентів.

Індивідуальні завдання:

13	За алфавітом	З парною сумою оцінок
----	--------------	-----------------------

Таблиця 1 — варіант індивідуального завдання

main.cpp

```
#include <iostream>
#include <string>
#include <cstring>
#include <algorithm>
using namespace std;

const int SUBJECTS = 5;

struct Student {
    unsigned int id;
    string surname;
    int grades[SUBJECTS];
};

void printStudent(const Student& info) {
    cout << "ID: " << info.id << ", Surname: " << info.surname << ", Grades: ";
    for (int g : info.grades) cout << g << " ";
    cout << "\n";
}

int gradeSum(const Student* info) {
    int sum = 0;
    for (int grade : info->grades)
        sum += grade;
    return sum;
}

bool evenSum(const Student* info) {
    return gradeSum(info) % 2 == 0;
}

void sortStudents(Student arr[], int size) {
    sort(arr, arr + size, [](const Student& a, const Student& b){
        return a.surname < b.surname;
    });
}
```

```

void printAll(const Student arr[], int size) {
    cout << "All students:\n";
    for (int i = 0; i < size; ++i) printStudent(arr[i]);
}

void searchEvenSum(const Student arr[], int size) {
    cout << "\nStudents with even sum of grades:\n";
    for (int i = 0; i < size; ++i) {
        if (evenSum(&arr[i])) {
            printStudent(arr[i]);
        }
    }
}

int main() {
    Student arr[] = {
        {1001, "Pisarenko", {90, 80, 85, 75, 70}},
        {1002, "Makarenko", {88, 92, 80, 76, 84}},
        {1003, "Shevchenko", {99, 91, 87, 93, 95}}
    };
    const int size = sizeof(arr) / sizeof(Student);

    sortStudents(arr, size);
    printAll(arr, size);
    searchEvenSum(arr, size);
    return 0;
}

```

Додатковий опис:

1.Результат при коректній роботі умови:

```

maksim@192 1.2 % cd ../1.3
maksim@192 1.3 % g++ -std=c++17 main.cpp -o main && ./main

All students:
ID: 1002, Surname: Makarenko, Grades: 88 92 80 76 84
ID: 1001, Surname: Pisarenko, Grades: 90 80 85 75 70
ID: 1003, Surname: Shevchenko, Grades: 99 91 87 93 95

Students with even sum of grades:
ID: 1002, Surname: Makarenko, Grades: 88 92 80 76 84
ID: 1001, Surname: Pisarenko, Grades: 90 80 85 75 70

```

Рисунок 3 — результат коректної роботи програми

1.4 Робота з однобічно зв'язаним списком чисел

Написати програму, яка забезпечує файлове введення та виведення і включає індивідуальне завдання лабораторної роботи № 5 курсу "Основи програмування (частина 1)". Слід реалізувати такі дії:

- визначення константи (**n**) яка визначає кількість стовпців двовимірного масиву
- відкриття файлу для читання (файл повинен бути підготовлений за допомогою текстового редактора)
- читання цілих чисел до кінця файлу і зберігання їх у зв'язаному списку
- створення двовимірного масиву в динамічній пам'яті; кількість рядків повинна бути обчислена на основі кількості зчитаних з файлу значень та визначеної кількості стовпців
- заповнення двовимірного масиву рядок за рядком; відсутні елементи останнього рядка повинні бути заповнені нулями
- видалення елементів зв'язаного списку з динамічної пам'яті
- реалізація індивідуального завдання лабораторної роботи № 5 курсу "Основи програмування (частина 1)".
- зберігання результатів в новому файлі
- видалення масивів операцією **delete**.

header.h

```
#ifndef HEADER_H
#define HEADER_H

#include <iostream>
#include <fstream>
#include <cmath>
#include <ctime>
using namespace std;

const int n = 5;

struct Node {
    int value;
    Node* next;
};

void append(Node*& head, int value);
int count(Node* head);
void clear(Node*& head);
```

```

int** create_matrix(int rows, int n);
void fill_matrix_from_list(Node* head, int** matrix, int rows, int n);
void replace_positive_with_sqrt(int** matrix, int rows, int n);
int* max_in_rows(int** matrix, int rows, int n);
void print_matrix(int** matrix, int rows, int n, ostream& out);
void delete_matrix(int** matrix, int rows);
void print_array(int* arr, int size, ostream& out);

#endif

```

main.cpp

```

#include "header.h"

int main() {
    Node* list = nullptr;

    ifstream fin("res/input.txt");
    if (!fin) {
        cerr << "Не вдалося відкрити файл input.txt" << endl;
        return 1;
    }

    int num;
    while (fin >> num) {
        append(list, num);
    }
    fin.close();

    int total = count(list);
    int rows = (total + n - 1) / n;

    int** matrix = create_matrix(rows, n);
    fill_matrix_from_list(list, matrix, rows, n);

    replace_positive_with_sqrt(matrix, rows, n);
    int* max_arr = max_in_rows(matrix, rows, n);

    ofstream fout("res/output.txt");
    fout << "Оброблена матриця:\n";
    print_matrix(matrix, rows, n, fout);

    fout << "\nМаксимальні елементи по кожному рядку:\n";
    print_array(max_arr, rows, fout);
    fout.close();
}

```



```

delete_matrix(matrix, rows);
delete[] max_arr;
clear(list);

cout << "Результат записано у файл output.txt\n";
return 0;
}

```

Додатковий опис:

1.Результат при коректній роботі умови:

The screenshot shows a code editor with three panels. The left panel displays `input.txt` with the following content:

```

1 25 -3 4 16 9
2 -2 100 -5 49 1
3 0 -7 11
4

```

The right panel displays `output.txt` with the following content:

```

1 Оброблена матриця:
2 5 -3 2 4 3
3 -2 10 -5 7 1
4 0 -7 3 0 0
5
6 Максимальні елементи по кожному р
7 5 10 3
8

```

The bottom panel is a terminal window showing the execution of the program. The command executed is `make && ./program && make clean`. The output shows the compilation of various C++ files using `clang++` and the execution of the resulting `program` binary. The terminal output includes the message "Результат записано у файл output.txt" and the removal of object files.

Рисунок 3 — результат коректної роботи програми

1.5 Робота з двобічно зв'язаним списком даних про студентів (додаткове завдання)

Реалізувати завдання 1.3, розташувавши дані про студентів у динамічній пам'яті, а відповідні вказівники розташувати у двобічно зв'язаному списку. Реалізувати сортування методом вибору. Не використовувати масивів. Коректно видалити дані з динамічної пам'яті.

main.cpp

```
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

const int SUBJECTS = 5;

struct Student {
    unsigned int id;
    string surname;
    int grades[SUBJECTS];
};

struct Node {
    Student* data;
    Node* prev;
    Node* next;
};

void addNode(Node*& head, Node*& tail, Student* student) {
    Node* newNode = new Node;
    newNode->data = student;
    newNode->prev = tail;
    newNode->next = nullptr;

    if (tail) {
        tail->next = newNode;
    } else {
        head = newNode;
    }
    tail = newNode;
}

void printStudent(const Student& info) {
    cout << "ID: " << info.id << ", Surname: " << info.surname << ", Grades: ";
```

```

        for (int g : info.grades) cout << g << " ";
        cout << "\n";
    }

    int gradeSum(const Student* info) {
        int sum = 0;
        for (int grade : info->grades)
            sum += grade;
        return sum;
    }

    bool evenSum(const Student* info) {
        return gradeSum(info) % 2 == 0;
    }

    void sortStudents(Node* head) {
        Node* current = head;
        while (current) {
            Node* minNode = current;
            Node* temp = current->next;
            while (temp) {
                if (temp->data->surname < minNode->data->surname) {
                    minNode = temp;
                }
                temp = temp->next;
            }
            swap(current->data, minNode->data);
            current = current->next;
        }
    }

    void printAll(Node* head) {
        cout << "All students:\n";
        Node* current = head;
        while (current) {
            printStudent(*(current->data));
            current = current->next;
        }
    }

    void searchEvenSum(Node* head) {
        cout << "\nStudents with even sum of grades:\n";
        Node* current = head;
        while (current) {
            if (evenSum(current->data)) {
                printStudent(*(current->data));
            }
        }
    }

```

```

        current = current->next;
    }
}

void deleteList(Node*& head) {
    Node* current = head;
    while (current) {
        Node* next = current->next;
        delete current->data;
        delete current;
        current = next;
    }
    head = nullptr;
}

int main() {
    Node* head = nullptr;
    Node* tail = nullptr;

    addNode(head, tail, new Student{1001, "Pisarenko", {90, 80, 85, 75, 70}});
    addNode(head, tail, new Student{1002, "Makarenko", {88, 92, 80, 76, 84}});
    addNode(head, tail, new Student{1003, "Shevchenko", {99, 91, 87, 93, 95}});

    sortStudents(head);
    printAll(head);
    searchEvenSum(head);

    deleteList(head);
    return 0;
}

```

Додатковий опис:

1.Результат при коректній роботі умови:

```

maksim@192 1.5 % g++ -std=c++17 main.cpp -o main && ./main

```

All students:

ID: 1002, Surname: Makarenko, Grades: 88 92 80 76 84

ID: 1001, Surname: Pisarenko, Grades: 90 80 85 75 70

ID: 1003, Surname: Shevchenko, Grades: 99 91 87 93 95

Students with even sum of grades:

ID: 1002, Surname: Makarenko, Grades: 88 92 80 76 84

ID: 1001, Surname: Pisarenko, Grades: 90 80 85 75 70

Рисунок 3 — результат коректної роботи програми

Висновок:

У ході виконання лабораторної роботи №2 було закріплено навички роботи зі структурами, переліками, масивами, динамічною пам'яттю та зв'язаними списками. Було реалізовано:

- перелік для представлення місяців року з перевантаженням операцій ++ і --;
- структуру для опису тривимірної точки та функцію для обчислення відстані між точками;
- структуру для представлення даних про студентів із функціями сортування (за прізвищем) та пошуку (студенти з парною сумою оцінок);
- роботу з однозв'язаним списком та динамічною матрицею: зчитування чисел з файлу, формування матриці, заміна додатних елементів на квадратні корені, пошук максимальних елементів у рядках;
- вивід результатів у файл.

Таким чином, було досягнуто поставленої мети лабораторної: набуття практичних навичок створення й обробки складних структур даних, а також організації взаємодії між різними типами даних і файлами.